

Interactive / complex / 7

IC 1	query	Interactive / complex / 7			
IC 2	title	Recent likers			
IC 3	pattern	<pre> graph TD person[person: Person] -- «opt» knows --> person person -- hasCreator --> message[message: Message] friend[friend: Person] -- likes --> message subgraph person_entity [person: Person] id_person[id = \$personId] end subgraph friend_entity [friend: Person] id_friend[id] first_name[firstName] last_name[lastName] end subgraph message_entity [message: Message] id_message[id] content_image[content / imageFile] end </pre>			
IC 4					
IC 5					
IC 6					
IC 7					
IC 8	description	Given a start Person with ID <code>\$personId</code>, find the most recent <code>likes</code> on any of start Person's Messages. Find Persons that liked (<code>likes</code> edge) any of start Person's Messages, the Messages they liked most recently, the creation date of that like, and the latency in minutes (<code>minutesLatency</code>) between creation of Messages and like. Additionally, for each Person found return a flag indicating (<code>isNew</code>) whether the liker is a friend of start Person. In case that a Person liked multiple Messages at the same time, return the Message with lowest identifier. <i>Validation rule:</i> Depending on whether the system-under-test supports leap seconds or uses UTC-SLS (UTC with Smoothed Leap Seconds), a difference of 1 minute can occur between the <code>minutesLatency</code> results of two correct implementations when the time interval includes June 30, 2012, when there was a leap second. Therefore, the <code>minutesLatency</code> value is validated using a tolerance of 1 minute.			
IC 10					
IC 11					
IC 12					
IC 13					
IC 14v1	params	1	<code>\$personId</code>	ID	
IC 14v2					
result	1	<code>friend.id</code>	ID	R	<code>friend.id = personId</code> is allowed
	2	<code>friend.firstName</code>	String	R	
	3	<code>friend.lastName</code>	String	R	
	4	<code>likes.creationDate</code>	DateTime	R	
	5	<code>message.id</code>	ID	R	
	6	<code>message.content</code> or <code>message.imageFile</code> (for photos)	Text	R	
	7	<code>minutesLatency</code>	32-bit Integer	C	Duration between the creation of the Message and the creation of the like, in minutes.
	8	<code>isNew</code>	Boolean	C	False if person and friend know each other, True otherwise
sort	1	<code>likes.creationDate</code>	↓		
	2	<code>friend.id</code>	↑		
limit	20				
CPs	2.2, 2.3, 3.3, 5.1, 8.1, 8.3				
relevance	This query looks for paths of length two, starting from a given Person, moving to its published messages and then to Persons who liked them. It tests several aspects related to join optimization, both at query optimization plan level and execution engine level. On the one hand, many of the columns needed for the projection are only needed in the last stages of the query, so the optimizer is expected to delay the projection until the end. This query implies accessing two-hop data, and as a consequence, index accesses are expected to be scattered. We expect to observe variate cardinalities, depending on the characteristics of the input parameter, so properly selecting the join operators will be crucial. This query has a lot of correlated sub-queries, so it is testing the ability to flatten the query execution plans.				